4.1 Notes

Site: Eduvos LMS
Course: ITLCA3 (First Block)
Book: 4.1 Notes
Printed by: George Conradie
Date: Monday, 2 March 2026, 7:30 PM

Table of contents

1. Learning outcomes
2. Number Systems
- 2.1. Binary Number Systems
- 2.2. One's Complements
- 2.3. Two's Complement
- 2.4. Octal System
- 2.5. Hexadecimal Number System
- 2.6. ASCII Code

1. Learning outcomes

By the end of this lesson, you should be able to:

- Understand common number systems, conversion and common character encodings

Prescribed Reading

Frank Petruzella, Programmable Logic Controllers.
McGraw-Hill Higher Education, 5th Edition, 2017. Print
ISBN: 9780073373843

Chapter 3, Section 3.1 - 3.11

2. Number Systems

A **number system** is a mathematical framework used to represent and express numbers in a structured manner. It defines a set of symbols (digits) and rules for performing arithmetic operations like addition, subtraction, multiplication, and division.

In general, PLCs work on binary numbers in one form or another; these are used to represent various codes or quantities.

The **decimal system**, which is most common to us, has a base of 10. The radix or base of a number system determines the total number of different symbols or digits used by that system.

The four common number systems are

- **Decimal**: uses ten symbols - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- **Binary**: uses two symbols - 0,1
- **Hexadecimal**: uses sixteen symbols - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- **Octal**: uses eight symbols - 0, 1, 2, 3, 4, 5, 6, 7

Table 3-1 Number System Comparisons

Decimal	Octal	Hexadecimal	Binary
0	0	0	0
1	1	1	1
2	2	2	10
3	3	3	11
4	4	4	100
5	5	5	101
6	6	6	110
7	7	7	111
8	10	8	1000
9	11	9	1001
10	12	A	1010
11	13	B	1011
12	14	C	1100
13	15	D	1101
14	16	E	1110
15	17	F	1111
16	20	10	10000
17	21	11	10001
18	22	12	10010
19	23	13	10011
20	24	14	10100

2.1. Binary Number Systems

The binary system uses the number 2 as the base. The only allowable digits are 0 and 1. With digital circuits, it is easy to distinguish between two voltage levels (i.e., +5 V and 0 V), which can be related to the binary digits 1 and 0.

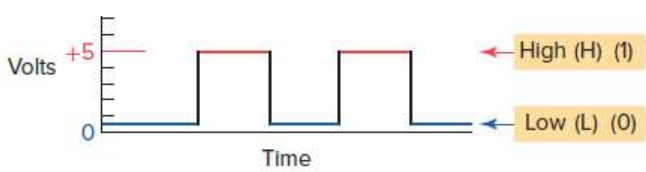

Figure 3-2 Digital signal waveform.

Decimal to Binary Number Conversion

Converting a **decimal number** (base-10) to a **binary number** (base-2) involves repeatedly dividing the decimal number by 2 and recording the remainders. The final binary number is obtained by **reading the remainders in reverse order**.

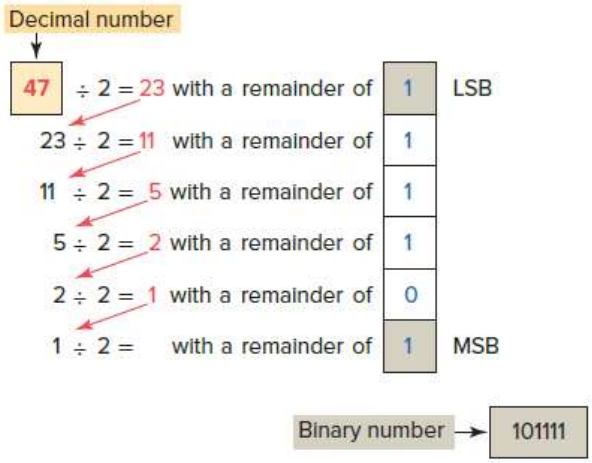

Figure 3-6 Converting a decimal number to a binary number.

Binary to Decimal Conversion

Converting a **binary number** (base-2) to a **decimal number** (base-10) involves multiplying each binary digit by 2 raised to the power of its position and summing the results.

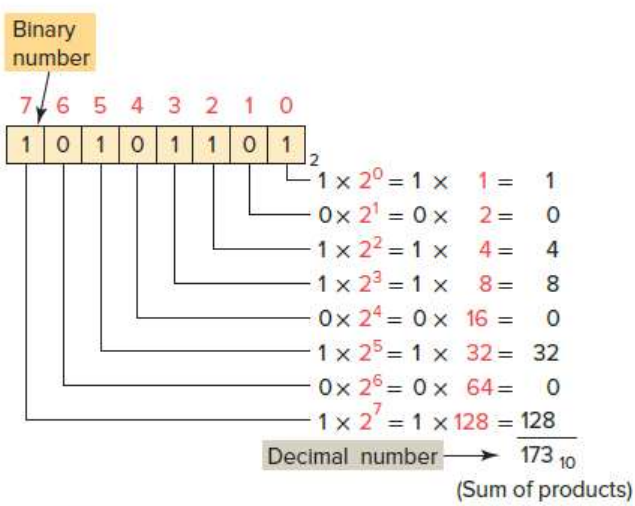

Figure 3-3 Converting a binary number to a decimal number.

2.2. One's Complements

The 1's complement method is a binary number representation technique primarily used in computer systems for signed number representation and arithmetic operations. It is commonly applied in older computer architectures and networking.

The 1's complement of a binary number is obtained by **flipping all the bits** (i.e., changing 1 to 0 and 0 to 1). It is mainly used to represent **negative numbers** in a signed binary number system.

For example, in an 8-bit system, numbers are represented as:

+5 00000101

-5 11111010 (1's complement of 00000101)

Subtraction can thus be done using the 1's complement approach

1. Convert the given numbers into binary (if not already in binary).
2. Find the 1's complement of the subtrahend (the number to be subtracted).
3. Add the 1's complement of the subtrahend to the minuend.
4. Check if there is a carry:
    - If there is a carry, add it back to the least significant bit (LSB) to get the final result.
    - If there is no carry, the result is in negative form and is already in 1's complement representation (convert it back to its negative decimal equivalent by taking its 1's complement and adding a negative sign).

Example:

87 (base 10) - 65 (base 10).

First, convert the two numbers from decimal to binary.

87 = 1010111

65 = 1000001

1'st complement method:

Find the first complement of subtrahend (65)

1's complement (1000001) = 0111110

Add it with the minuend (87):

1010111

+0111110

10010101

The result has a carry-over. It must be dropped resulting in 0010101. One must be added to the last bit. 0010101 + 1 = **0010110 (base 2)** which convert back to decimal yield **22 (base 10)**

2.3. Two's Complement

The **two's complement** method is the most widely used system for representing signed binary numbers and performing arithmetic operations in modern digital computers. It simplifies subtraction and eliminates the issue of having both +0 and -0, which exists in 1's complement.

The **two's complement** of a binary number is obtained by:

1. **Taking the 1's complement** (inverting all bits).
2. **Adding 1** to the inverted number.

This method is used to represent negative numbers and perform subtraction using binary addition.

E.g. in an 8-bit system,

+5 00000101

-5 11111011 (1's complement of 00000101 +1)

Subtraction can thus be done using the 2's complement approach

- **Convert the given numbers into binary** (ensure they have the same number of bits).
- **Find the two's complement of the subtrahend (the number being subtracted).**
  - Take the **1's complement** (invert all bits).
  - Add **1** to the result.
- **Add the minuend (the first number) and the two's complement of the subtrahend.**
- **Ignore overflow** (if any occurs, discard the extra carry bit).
- **Interpret the result:**
  - If the MSB (Most Significant Bit) is 0, the result is positive.
  - If the MSB is 1, the result is negative and already in two's complement form.

Example:

87 (base 10) - 65 (base 10).

The 2's complement of the subtrahend (65) must be found:

From the first complement of 65: 1's complement (1000001) = 0111110, we had 1 to obtain its 2's complement. 0111110 + 1 = **0111111**

It must be added to the minuend (87): 1010111

1010111

+0111111

10010110

The result has a carry over. It must be dropped, **0010110** yielding the actual answer, which is a positive number: 0010110 (base 2) that we convert back to decimal:

22 (base 10)

2.4. Octal System

The octal numbering system, a base 8 system, is used because 8 data bits make up a byte of information that can be addressed. Octal is a convenient means of handling large binary numbers.

Decimal to Octal conversion

Converting a decimal number (base-10) to an octal number (base-8) involves repeatedly dividing the decimal number by 8 and recording the remainders. The final octal number is obtained by reading the remainders in reverse order.

Octal to Decimal conversion

Converting an octal number (base-8) to a decimal number (base-10) involves multiplying each digit by 8 raised to the power of its position and summing the results.

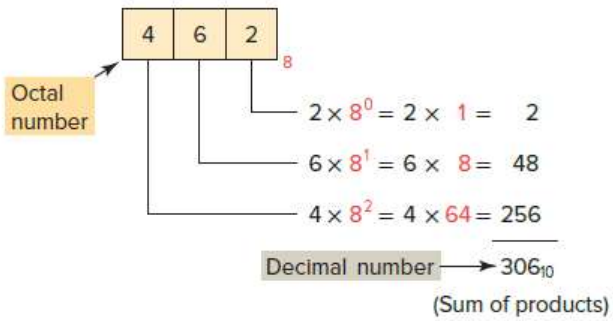

Figure 3-8 Converting an octal number to a decimal number.

2.5. Hexadecimal Number System

The hexadecimal (hex) numbering system is used in programmable controllers because a word of data consists of 16 data bits, or two 8-bit bytes. The hexadecimal system is a base 16 system, with A to F used to represent decimal numbers 10 to 15.

The techniques used when converting hexadecimal to decimal and decimal to hexadecimal are the same as those used for binary and octal.

Decimal to Hexadecimal

Converting a decimal number (base-10) to a hexadecimal number (base-16) involves repeatedly dividing the decimal number by 16 and recording the remainders. The final hexadecimal number is obtained by reading the remainders in reverse order.

Hexadecimal to Decimal

Converting a hexadecimal number (base-16) to a decimal number (base-10) involves multiplying each digit by 16 raised to the power of its position and summing the results.

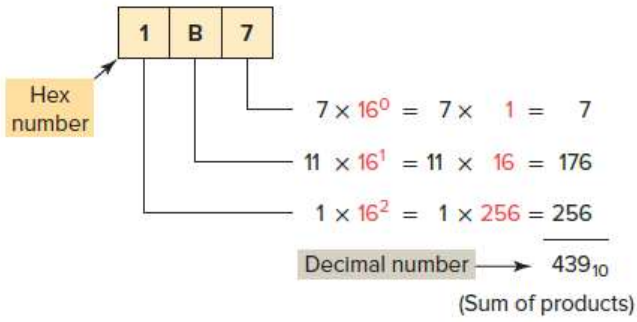

Figure 3-10 Converting a hexadecimal number to a decimal number.

2.6. ASCII Code

ASCII stands for American Standard Code for Information Interchange. It is an alphanumeric code because it includes letters as well as numbers. The characters accessed by the ASCII code include 10 numeric digits; 26 lowercase and 26 uppercase letters of the alphabet; and about 25 special characters, including those found on a standard typewriter.

Table 3-8 Partial Listing of ASCII Code

Character	7-Bit ASCII	Character	7-Bit ASCII
A	100 0001	X	101 1000
B	100 0010	Y	101 1001
C	100 0011	Z	101 1010
D	100 0100	0	011 0000
E	100 0101	1	011 0001
F	100 0110	2	011 0010
G	100 0111	3	011 0011
H	100 1000	4	011 0100
I	100 1001	5	011 0101
J	100 1010	6	011 0110
K	100 1011	7	011 0111
L	100 1100	8	011 1000
M	100 1101	9	011 1001
N	100 1110	blank	010 0000
O	100 1111	.	010 1110
P	101 0000	,	010 1100
Q	101 0001	+	010 1011
R	101 0010	−	010 1101
S	101 0011	#	010 0011
T	101 0100	(	010 1000
U	101 0101	%	010 0101
V	101 0110	=	011 1101
W	101 0111		

It is used to interface the PLC CPU with alphanumeric keyboards and printers. This seven-bit code provides all possible combinations of characters used when communicating with peripherals or interfaces in a PLC system.